
Perceptrón multicapa

¿Por qué varias capas?

Demostramos que el perceptrón clasifica bien, utilizando un conjunto de datos donde las clases estaban separadas por un límite lineal. Pero no siempre tenemos tanta suerte. Allá por 1969, se sabía que el perceptrón no podía clasificar el problema XOR.

La gente descubrió que este problema se puede superar ‘apilando’ perceptrones uno sobre el otro para que las entradas de uno de ellos sean las salidas de los de abajo. De ahí el nombre de **perceptrones multicapa, PMC (multi-layer perceptrons)**. Las capas entre la entrada y la salida se denominan capas ocultas.

Una red con dos o más capas ocultas se denomina red neuronal profunda.

Los primeros PMC se entrenaron utilizando un algoritmo de retropropagación. Sin embargo, hay muchos otros ahora. Demostraremos el uso del perceptrón multicapa usándolo en los datos del iris.

Preparando los datos

¿Cuántos casos tenemos para cada clase (especie)?

```
> aggregate(cbind(n.cases = Sepal.Length) ~ Species, iris, length)
  Species n.cases
1  setosa      50
2 versicolor  50
3 virginica   50
```

Hay un número igual de casos en cada clase. Preparemos nuestra muestra para que elijamos un número igual de cada clase.

```
> set.seed(18121842)
> iris.setosa <- iris[iris$Species == 'setosa',]
> iris.versicolor <- iris[iris$Species == 'versicolor',]
> iris.virginica <- iris[iris$Species == 'virginica',]
>
> split_data <- function(N, p) {
+   stopifnot(p > 0 & p < 1)
+   n <- N * p
+   trn.index <- sample.int(N, n, replace = FALSE)
```

```

+ test.index <- setdiff(1:N, trn.index)
+ list(train = trn.index, test = test.index)
+ }
>
> setosa.split <- split_data(nrow(iris.setosa), 0.2)
> setosa.train <- iris.setosa[setosa.split[["train"]],]
> setosa.test <- iris.setosa[setosa.split[["test"]],]
>
> versicolor.split <- split_data(nrow(iris.versicolor), 0.2)
> versicolor.train <- iris.versicolor[versicolor.split[["train"]],]
> versicolor.test <- iris.versicolor[versicolor.split[["test"]],]
>
> virginica.split <- split_data(nrow(iris.virginica), 0.2)
> virginica.train <- iris.virginica[virginica.split[["train"]],]
> virginica.test <- iris.virginica[virginica.split[["test"]],]
>
> train.data <- rbind(setosa.train, versicolor.train, virginica.train)
> test.data <- rbind(setosa.test, versicolor.test, virginica.test)

> rm(
+ setosa.split,
+ versicolor.split,
+ virginica.split,
+ iris.setosa,
+ iris.versicolor,
+ iris.virginica
+ )
> rm(setosa.train, versicolor.train, virginica.train)
> rm(setosa.test, versicolor.test, virginica.test)

```

Entrenando la red - 1

Usamos la red de una sola capa oculta provista en el paquete 'nnet'.

Recordemos que en el perceptrón teníamos un hiperparámetro η llamado tasa de aprendizaje. Influye en la velocidad en la que cambian los pesos en cada iteración. Usamos la relación

$$w_{i+1} = w_i - \eta \frac{\partial E}{\partial w_i}$$

para actualizar pesos en descenso de gradiente.

A veces los pesos pueden llegar a ser extremos. Para evitar eso, aplicamos un parámetro de regularización λ llamado decaimiento. Con él, se modifica el descenso de gradiente a

$$w_{i+1} = w_i - \eta \frac{\partial E}{\partial w_i} - \eta \lambda w_i$$

Como en el caso del perceptrón, también podemos especificar el número máximo de iteraciones.

La capa oculta se especifica pasando su tamaño.

Entrenando la red - 2

La llamada de función para construir la red neuronal de una sola capa oculta es

```
> library(nnet)
>
> nn.1 <-
+ nnet(
+ Species ~ .,
+ data = train.data,
+ size = 2,
+ decay = 1e-5,
+ maxit = 50
+ )
# weights: 19
initial value 35.411445
iter 10 value 32.955229
iter 20 value 14.638913
iter 30 value 13.888475
iter 40 value 13.869530
final value 13.868609
converged
```

No hemos quitado ninguna característica de los datos porque no sabemos cuáles de ellos son útiles en una clasificación de 3 vías.

Se puede ver la red usando la función de resumen.

```
> summary(nn.1)
a 4-2-3 network with 19 weights
options were - softmax modelling decay=1e-05
b->h1 i1->h1 i2->h1 i3->h1 i4->h1
 0.02  1.00  4.49 -8.03 -3.67
b->h2 i1->h2 i2->h2 i3->h2 i4->h2
-0.20 -1.34 -0.28 -0.83 -0.53
b->o1 h1->o1 h2->o1
-6.14 14.48  0.09
b->o2 h1->o2 h2->o2
 3.48 -5.69 -0.12
b->o3 h1->o3 h2->o3
```

3.48 -7.95 0.30

Observamos que hay cuatro neuronas de entrada i1, i2, i3, i4, una para cada característica.

También hay una neurona de "sesgo" b.

Hay dos neuronas en la capa oculta h1 y h2, junto con un sesgo b.

Como era de esperar, hay tres capas de salida o1, o2, o3.

También se mencionan los pesos de una neurona a las demás en la capa siguiente.

La capa de salida se trata con una función softmax para convertirla en una probabilidad.

Probando la red

Al igual que otros modelos en R, también tenemos la función de predicción para redes neuronales.

```
> predicted.species <- predict(nn.1, test.data, type = "class")
> comparison <- data.frame(actual = test.data$Species, predicted = predicted.species)
> table(comparison)
```

	predicted	
actual	setosa	versicolor
setosa	40	0
versicolor	0	40
virginica	0	40

Confundimos a toda virginica con versicolor.

¿Ayuda aumentar el número de iteraciones?

Modelo con aumento del número de iteraciones

```
> nn.2 <-
+ nnet(
+ Species ~ .,
+ data = train.data,
+ size = 2,
+ decay = 1e-5,
+ maxit = 100
+ )
# weights: 19
initial value 37.118395
iter 10 value 3.354185
iter 20 value 0.356276
iter 30 value 0.070115
iter 40 value 0.062318
iter 50 value 0.055942
```

```

iter 60 value 0.054585
iter 70 value 0.052726
iter 80 value 0.040042
iter 90 value 0.033892
iter 100 value 0.031714
final value 0.031714
stopped after 100 iterations

```

```

> predicted.species <- predict(nn.2, test.data, type = "class")
> comparison <- data.frame(actual = test.data$Species, predicted = predicted.species)
> table(comparison)
      predicted
actual      setosa versicolor virginica
setosa      40      0      0
versicolor  0      40      0
virginica   0      12     28

```

Se hizo algo, pero no demasiado. ¿Qué tal aumentar las capas ocultas?

Modelo con más unidades en la capa oculta

```

> nn.3 <-
+ nnet(
+ Species ~ .,
+ data = train.data,
+ size = 4,
+ decay = 1e-5,
+ maxit = 50
+ )
# weights: 35
initial value 39.909614
iter 10 value 13.743053
iter 20 value 2.588907
iter 30 value 0.436368
iter 40 value 0.027481
iter 50 value 0.022585
final value 0.022585
stopped after 50 iterations
> predicted.species <- predict(nn.3, test.data, type = "class")
> comparison <- data.frame(actual = test.data$Species, predicted = predicted.species)
> table(comparison)
      predicted
actual      setosa versicolor virginica
setosa      40      0      0

```

versicolor	0	40	0
virginica	0	11	29

Agregar capas ocultas no supuso una gran diferencia.

Modelo con más unidades y más iteraciones

```
> nn.4 <-
+ nnet(
+ Species ~ .,
+ data = train.data,
+ size = 4,
+ decay = 1e-5,
+ maxit = 50
+ )
# weights: 35
initial value 35.989656
iter 10 value 9.590075
iter 20 value 0.263699
iter 30 value 0.072545
iter 40 value 0.065118
iter 50 value 0.054472
final value 0.054472
stopped after 50 iterations
> predicted.species <- predict(nn.4, test.data, type = "class")
> comparison <- data.frame(actual = test.data$Species, predicted = predicted.species)
> table(comparison)
      predicted
actual  setosa versicolor virginica
setosa    40      0      0
versicolor  0    40      0
virginica   0    11    29
```

No hay cambio.

Modelo con menos funciones

```
> nn.5 <-
+ nnet(
+ Species ~ Petal.Length + Petal.Width,
+ data = train.data,
+ size = 4,
+ decay = 1e-5,
```

```

+ maxit = 50
+ )
# weights: 27
initial value 36.393243
iter 10 value 11.117675
iter 20 value 0.566856
iter 30 value 0.127323
iter 40 value 0.065351
iter 50 value 0.055558
final value 0.055558
stopped after 50 iterations
> predicted.species <- predict(nn.5, test.data, type = "class")
> comparison <- data.frame(actual = test.data$Species, predicted = predicted.species)
> table(comparison)
      predicted
actual   setosa versicolor virginica
setosa    40         0         0
versicolor  0        40         0
virginica   0         7        33

```

La eliminación de características marcó una gran diferencia. ¿Podemos mejorar?

Modelo con menos características y más iteraciones

```

> nn.6 <-
+ nnet(
+ Species ~ Petal.Length + Petal.Width,
+ data = train.data,
+ size = 4,
+ decay = 1e-5,
+ maxit = 50
+ )
# weights: 27
initial value 39.684126
iter 10 value 8.099589
iter 20 value 0.208046
iter 30 value 0.082138
iter 40 value 0.071433
iter 50 value 0.066694
final value 0.066694
stopped after 50 iterations
> predicted.species <- predict(nn.6, test.data, type = "class")
> comparison <- data.frame(actual = test.data$Species, predicted = predicted.species)
> table(comparison)

```

actual \ predicted			
	setosa	versicolor	virginica
setosa	37	3	0
versicolor	0	40	0
virginica	0	7	33

Ningún cambio.

Conclusiones

Un PMC con una sola capa tiene muchos más hiperparámetros con los que jugar.

A veces, eliminar funciones da mejores resultados. En este caso, la visualización de los datos nos ayudó a seleccionar las características.